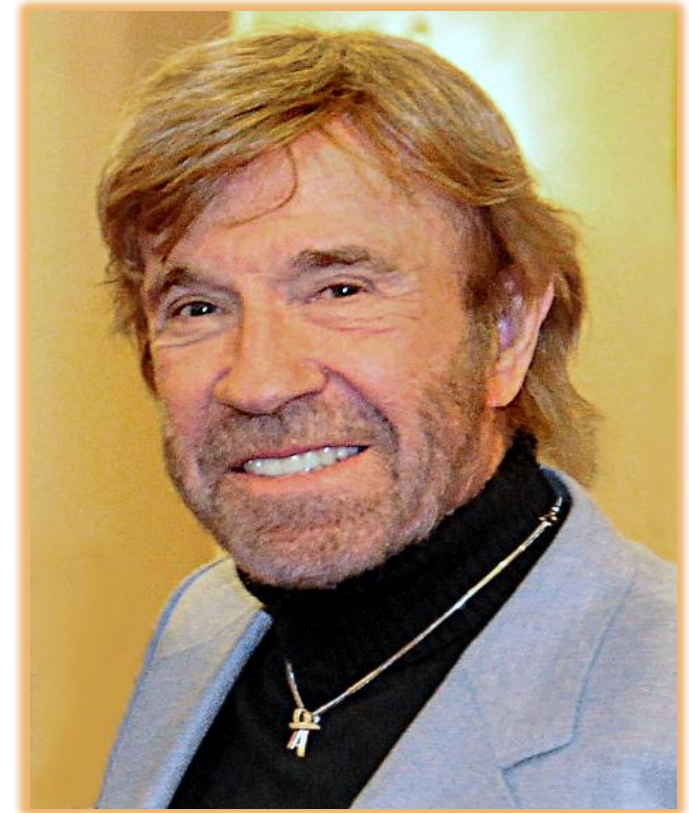


Chuck-Norris- Joke-Service



https://en.wikipedia.org/wiki/File:Chuck_Norris_May_2015.jpg

GRUPPE 4

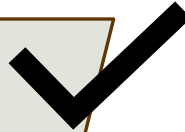
RALPH KALWEIT, NICO GEIS, MICHAEL NIEDERMAIER

Idee

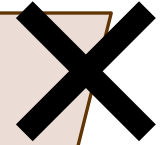
- Witze von Chuck Norris API abfragen
- Witze in eigener DB speichern (optional bearbeiten)
- Witze aus eigener DB öffentlich bereitstellen

Scope

Zufälligen Witz aus der
Datenbank abfragen



Witze gezielt abfragen

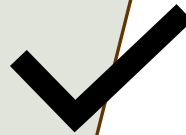


Scope

Zufälligen Witz aus der
Datenbank abfragen

Neue Witze erstellen:

- Selber eintippen
- Von öffentlicher API abfragen



Witze gezielt abfragen

Verwaltungsübersicht mit allen
Witzen für Admin



Scope

Zufälligen Witz aus der Datenbank abfragen

Witze gezielt abfragen

Neue Witze erstellen:

- Selber eintippen
- Von öffentlicher API abfragen

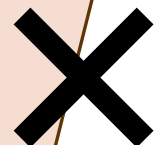
Verwaltungsübersicht mit allen Witzen für Admin

User Authentication: Anmeldung als Administrator



User-Management:

- Neuregistrierung
- Rollen (Admin vs. Standard)



Scope

Zufälligen Witz aus der Datenbank abfragen

Witze gezielt abfragen

Neue Witze erstellen:

- Selber eintippen
- Von öffentlicher API abfragen

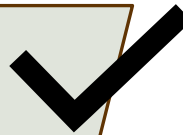
Verwaltungsübersicht mit allen Witzen für Admin

User Authentication: Anmeldung als Administrator

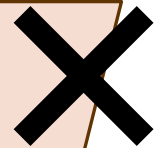
User-Management:

- Neuregistrierung
- Rollen (Admin vs. Standard)

GUI auf Englisch und Deutsch



Witz-Übersetzung



Scope

Zufälligen Witz aus der Datenbank abfragen

Witze gezielt abfragen

Neue Witze erstellen:

- Selber eintippen
- Von öffentlicher API abfragen

Verwaltungsübersicht mit allen Witzen für Admin

User Authentication: Anmeldung als Administrator

User-Management:

- Neuregistrierung
- Rollen (Admin vs. Standard)

GUI auf Englisch und Deutsch

Witz-Übersetzung



Light Theme und Dark Theme

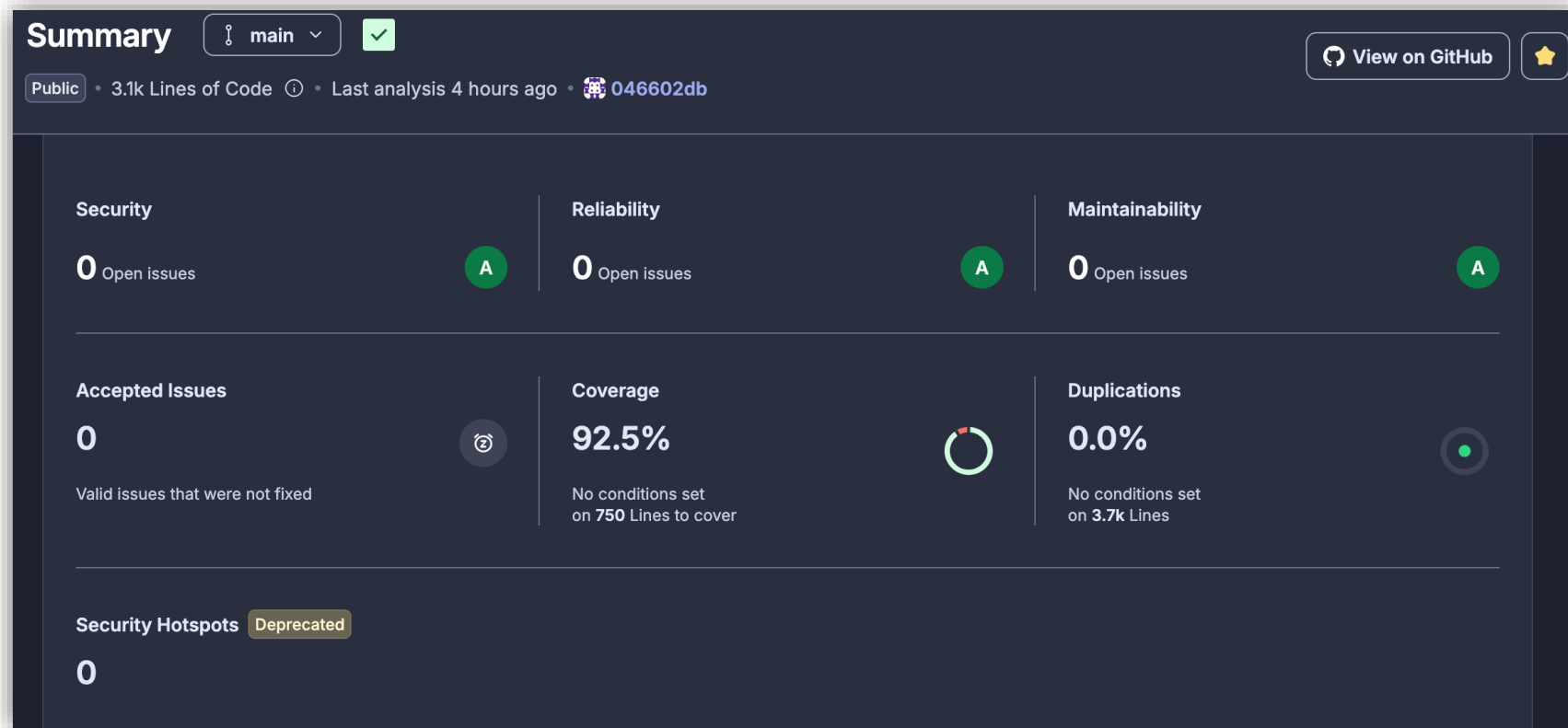
Health-Check



Live-Demo!

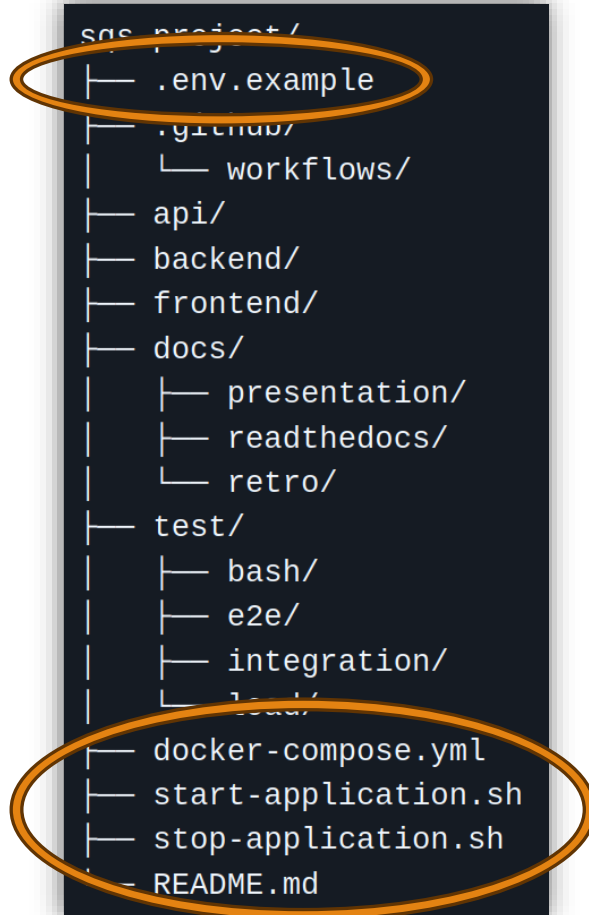
[Zu den Witzen](#)

SonarCloud



https://sonarcloud.io/summary/overall?id=MiChillDev_sqs-project&branch=main

Monorepo-Projektstruktur



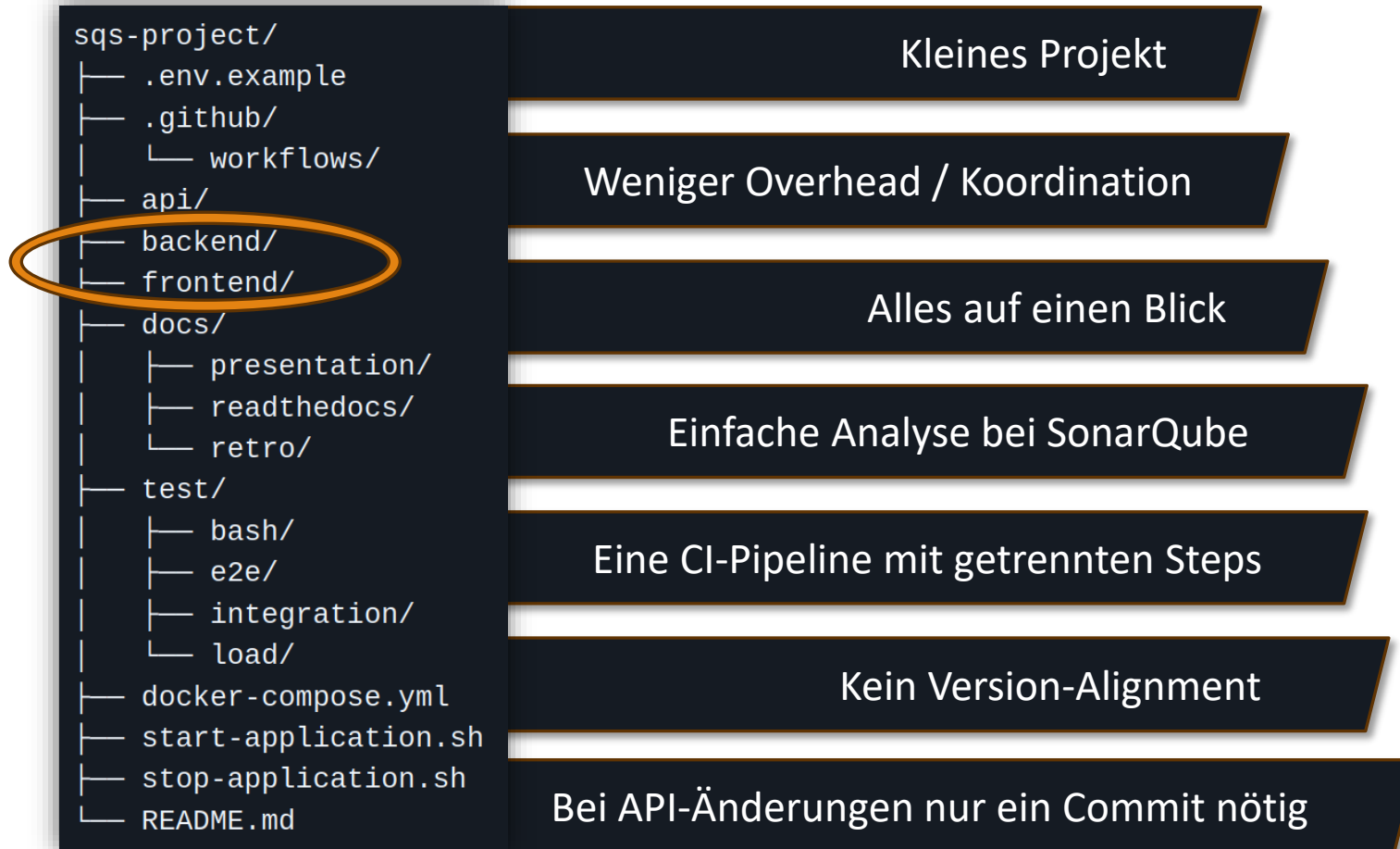
Monorepo-Projektstruktur

```
sqs-project/  
├── .env.example  
├── .github/  
│   └── workflows/  
├── api/  
├── backend/  
├── frontend/  
├── docs/  
│   ├── presentation/  
│   ├── readthedocs/  
│   └── retro/  
├── test/  
│   ├── bash/  
│   ├── e2e/  
│   ├── integration/  
│   └── load/  
├── docker-compose.yml  
├── start-application.sh  
├── stop-application.sh  
└── README.md
```

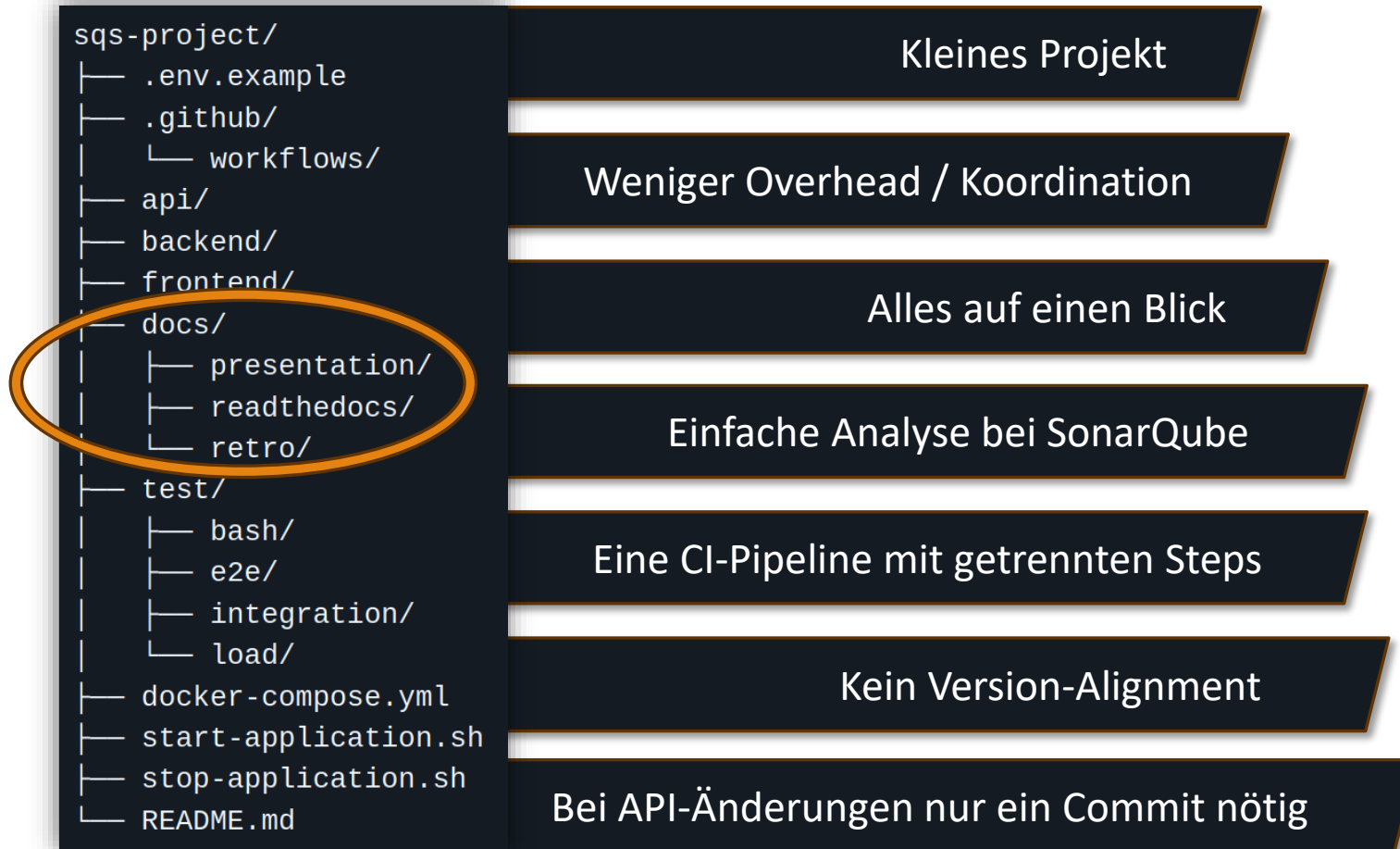
Monorepo-Projektstruktur

```
sqs-project/  
├── .env.example  
├── .github/  
├── workflows/  
├── api/  
├── backend/  
├── frontend/  
├── docs/  
│   ├── presentation/  
│   ├── readthedocs/  
│   └── retro/  
├── test/  
│   ├── bash/  
│   ├── e2e/  
│   ├── integration/  
│   └── load/  
├── docker-compose.yml  
├── start-application.sh  
├── stop-application.sh  
└── README.md
```

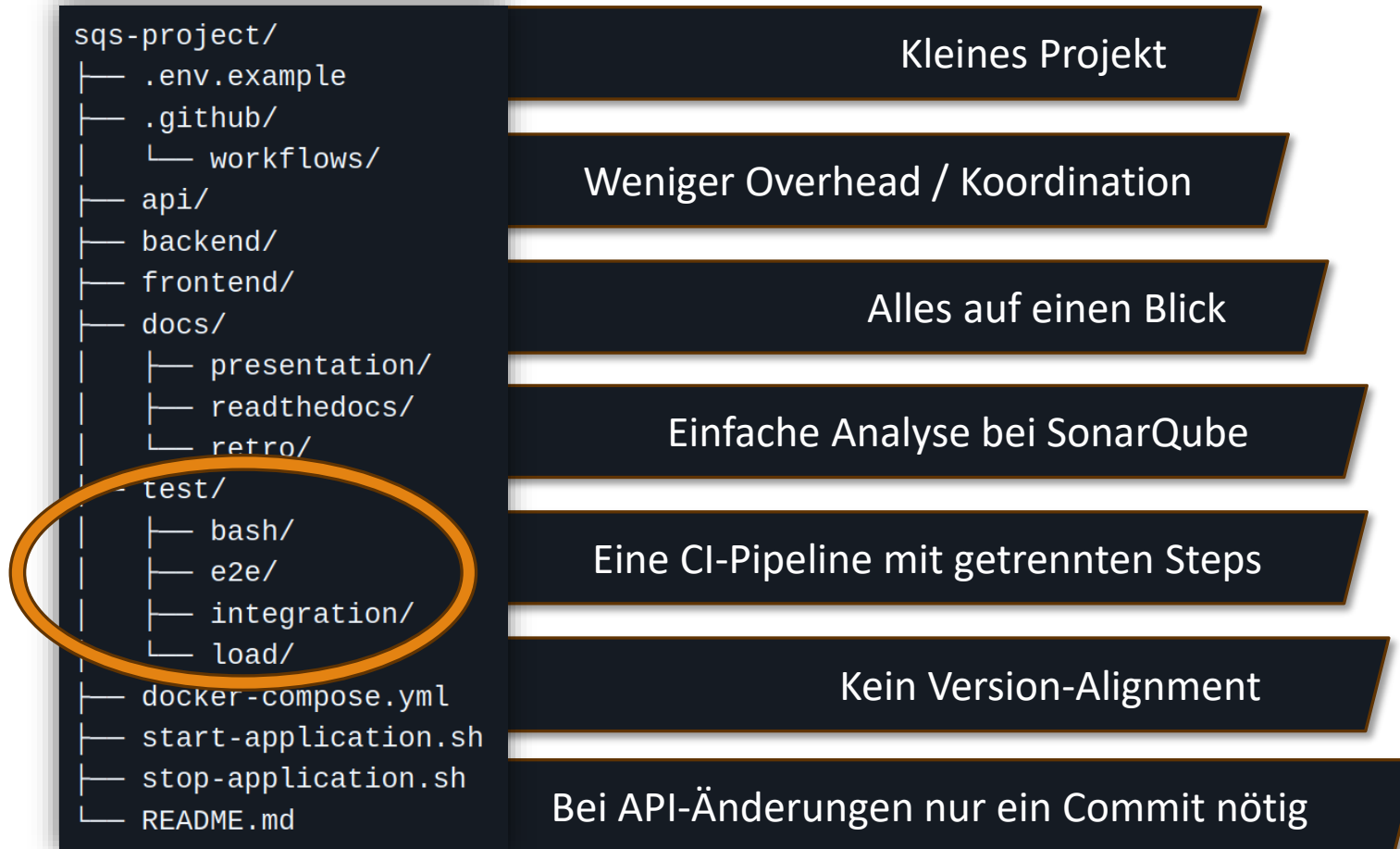
Monorepo-Projektstruktur



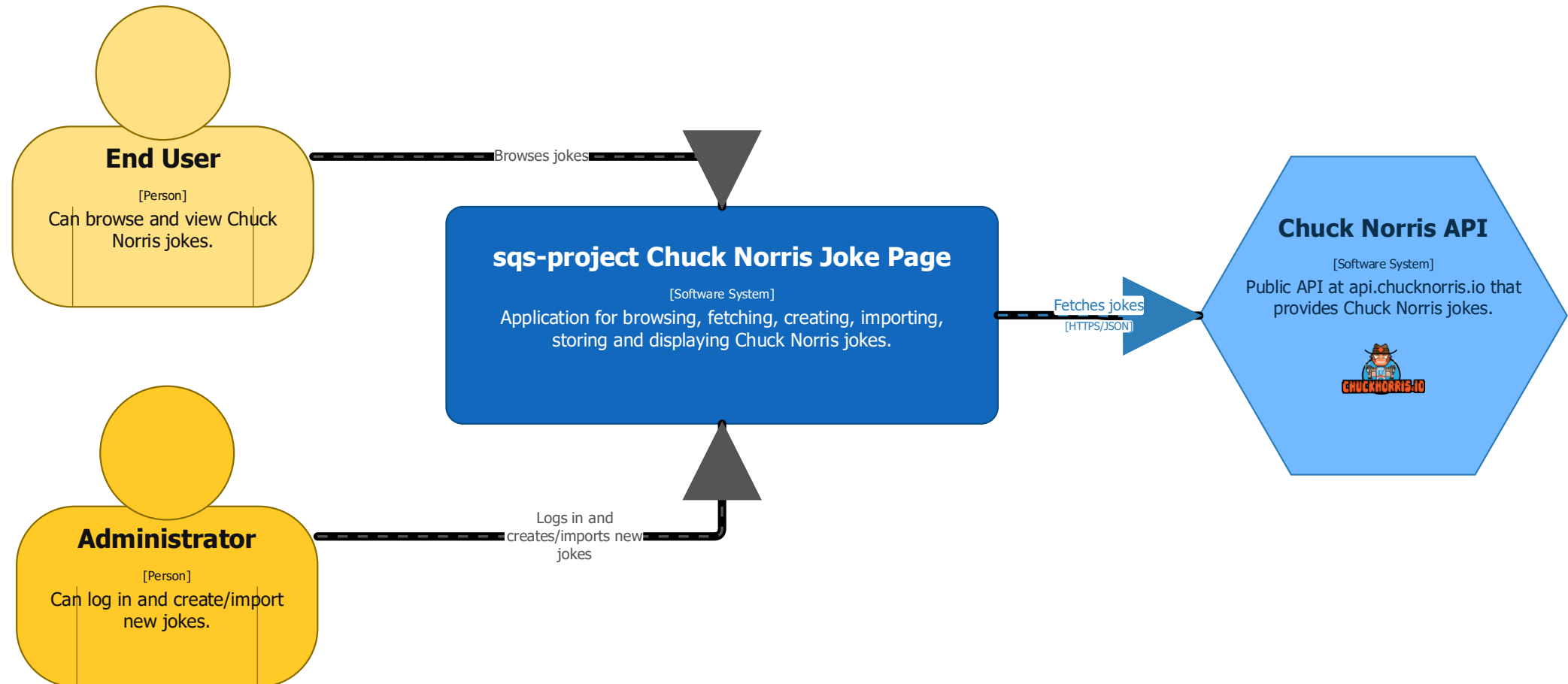
Monorepo-Projektstruktur



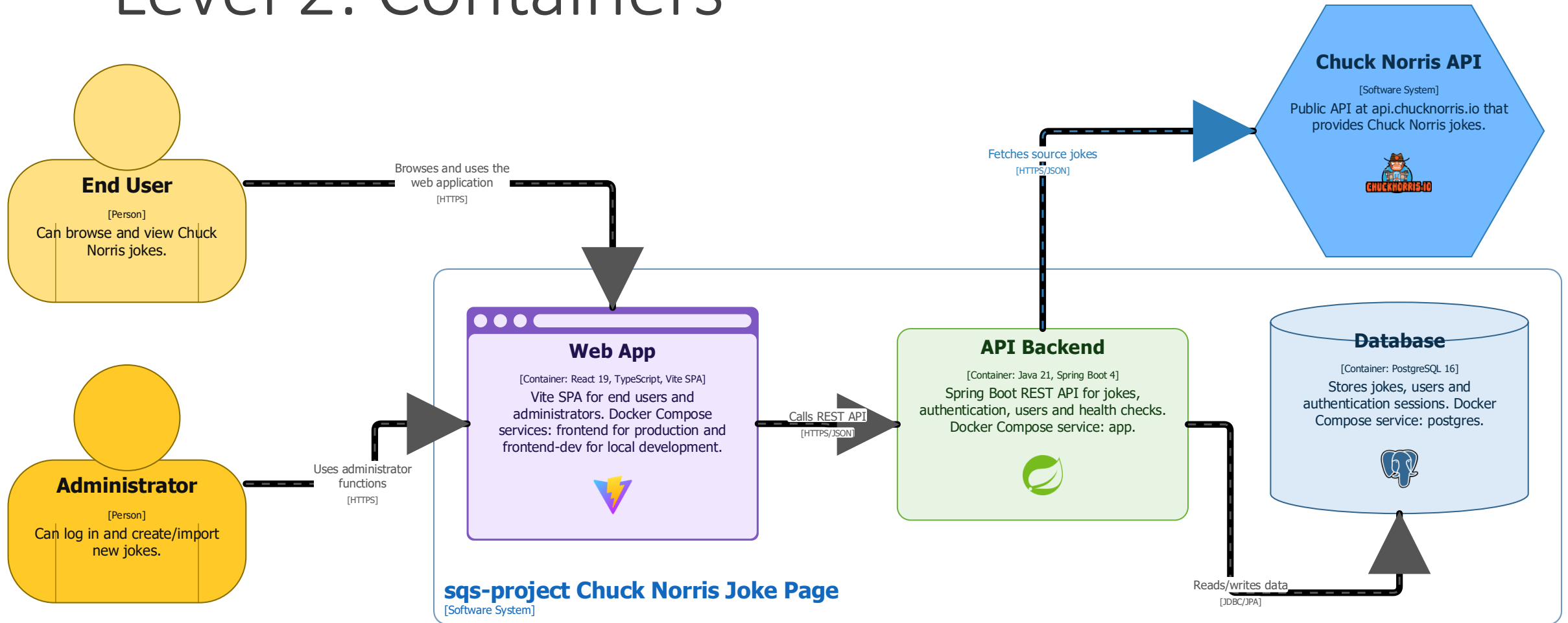
Monorepo-Projektstruktur



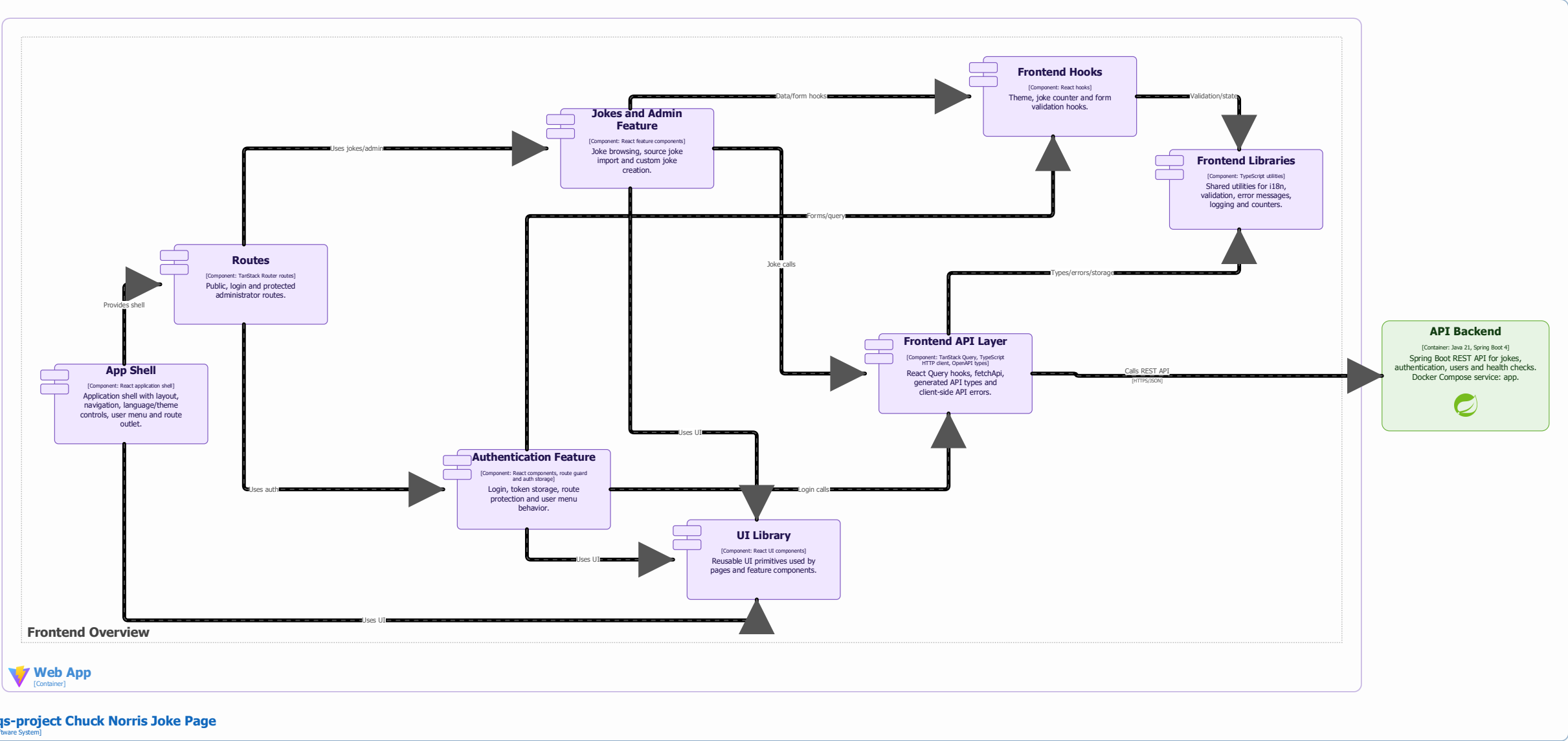
Level 1: Context



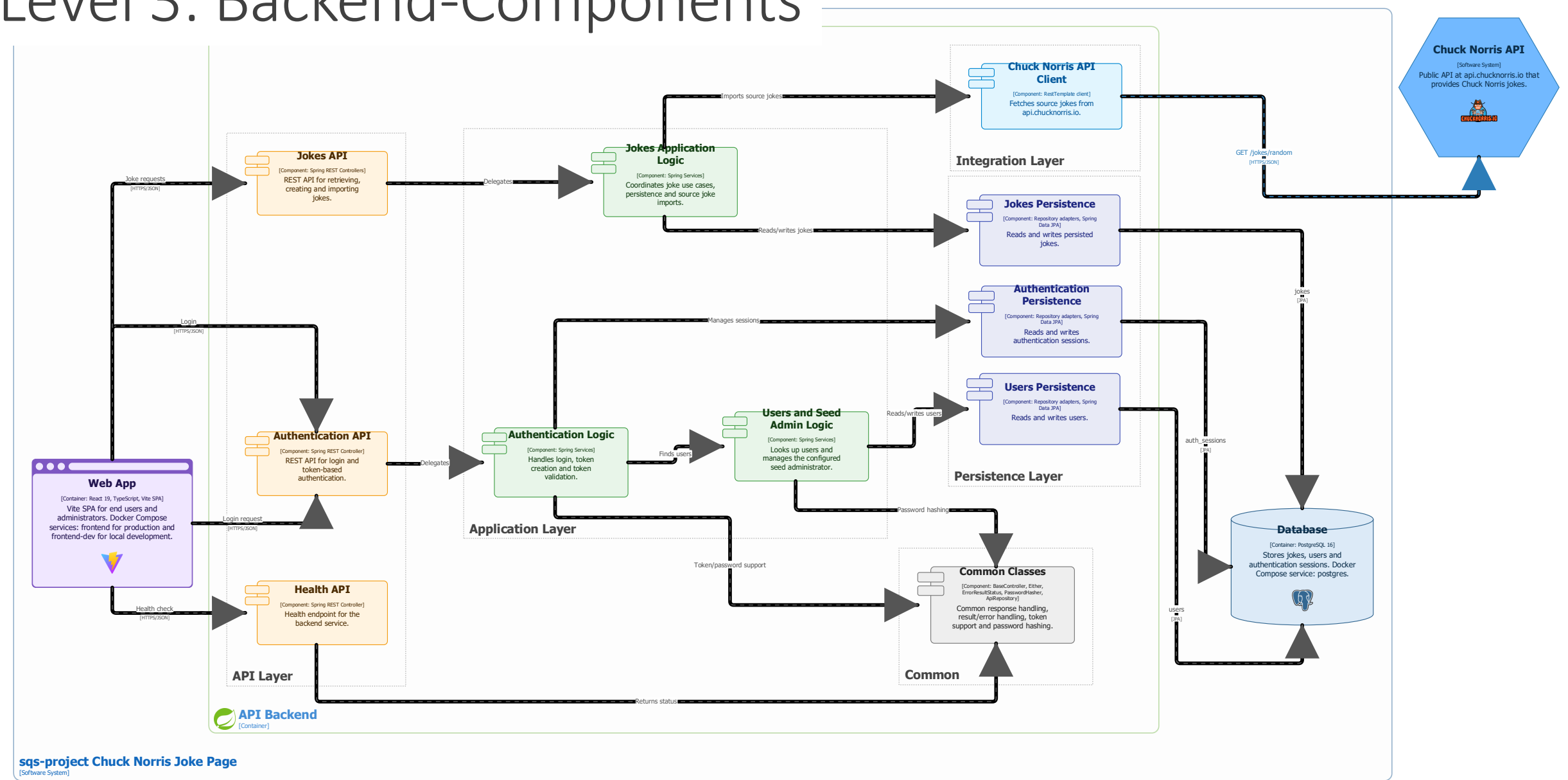
Level 2: Containers



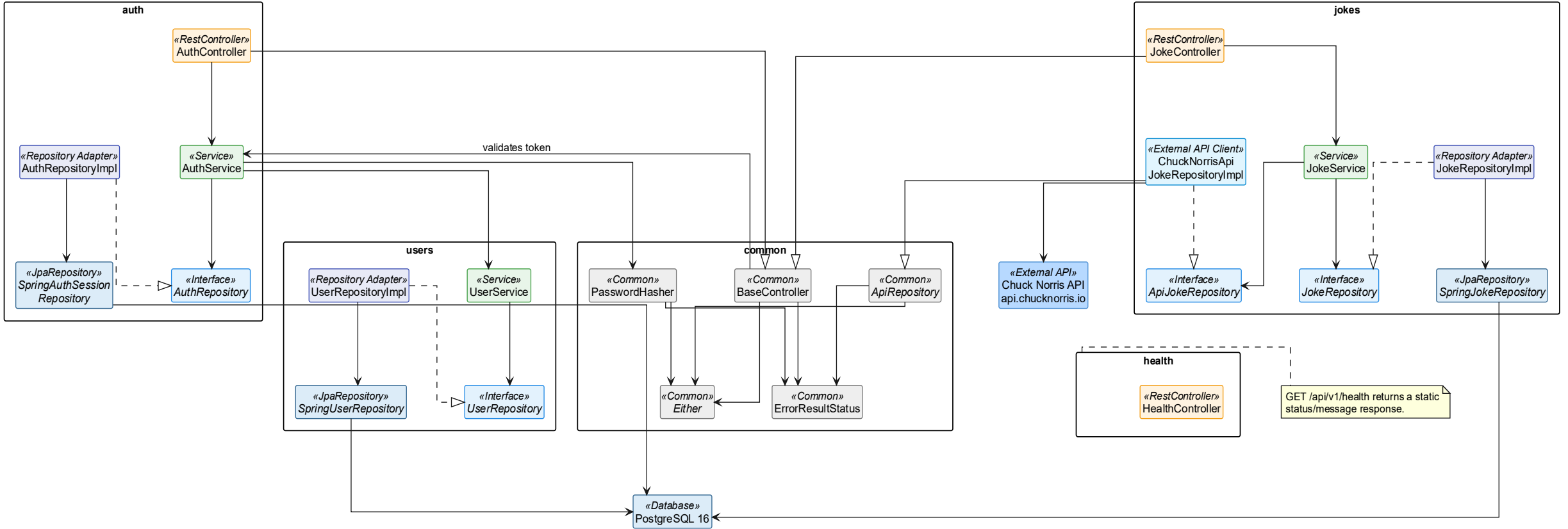
Level 3: Frontend-Components



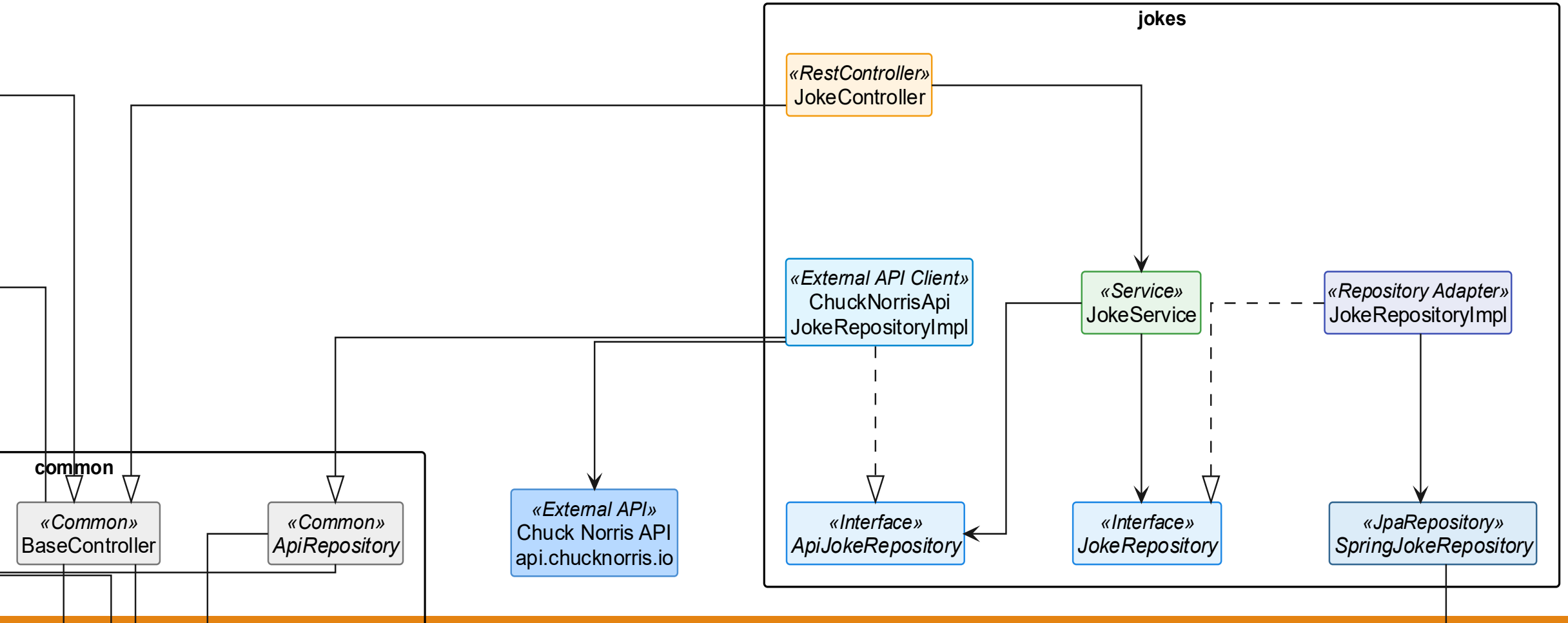
Level 3: Backend-Components



Level 4: Backend-Code



Level 4: Backend-Code



Unit Tests - Frontend

- Technologie: Vitest
- Erstellen eines Coverage Berichts mithilfe von v8

```
frontend/src/shared/lib/login-schema.ts

import { z } from 'zod';

export const loginSchema = z.object({
  username: z.string().min(1).max(255),
  password: z.string().min(1).max(255),
});

export type LoginFormValues = z.infer<typeof loginSchema>;
```

Implementierung

```
frontend/tests/unit/shared/lib/login-schema.test.ts

describe('loginSchema', () => {
  it('fails validation when username is empty', () => {
    const result = loginSchema.safeParse({ username: '', password: 'secret' });

    expect(result.success).toBe(false);
  });

  it('passes validation with valid username and password', () => {
    const result = loginSchema.safeParse({ username: 'alice', password: 'secret' });

    expect(result.success).toBe(true);
    if (result.success) {
      const data: LoginFormValues = result.data;
      expect(data.username).toBe('alice');
      expect(data.password).toBe('secret');
    }
  });
});
```

Unit tests

Unit Tests - Backend

- Technologie: JUnit
- Erstellen eines Coverage Berichts mit JaCoCo
- Tests für jedes Layer, mocking der darunterliegenden Layers mit Mockito

```
backend/src/test/java/com/chucknorris/auth/controller/AuthControllerTest.java

@Test
@DisplayName("returns 200 with token when credentials are valid")
void returns200WithToken() {
    TokenResponseDto dto = new TokenResponseDto("tok-1", LocalDateTime.MAX);
    when(authService.login(new LoginRequestDto("bob", "pw"))).thenReturn(Either.right(dto));

    ResponseEntity<TokenResponseDto> resp = controller.login(new LoginRequestDto("bob", "pw"));
    assertThat(resp.getStatusCode().value()).isEqualTo(200);
    assertThat(resp.getBody()).isNotNull();
    assertThat(resp.getBody().token()).isEqualTo("tok-1");
}
```

Architekturtests - Backend

- Technologie: UnitArch
- Integriert seamless in JUnit
- Controller, Service, Repository, RepositoryImpl müssen in entsprechenden Packages liegen
- Zugriff nur Controller -> Service -> Repository
- Controller, Service, RepositoryImpl, utils/** müssen eine Testklasse haben
- Klassen müssen entsprechende Annotierungen haben (z.B. @Service)

Architekturtests - Backend

```
backend/src/test/java/architecture/ArchitectureTest.java

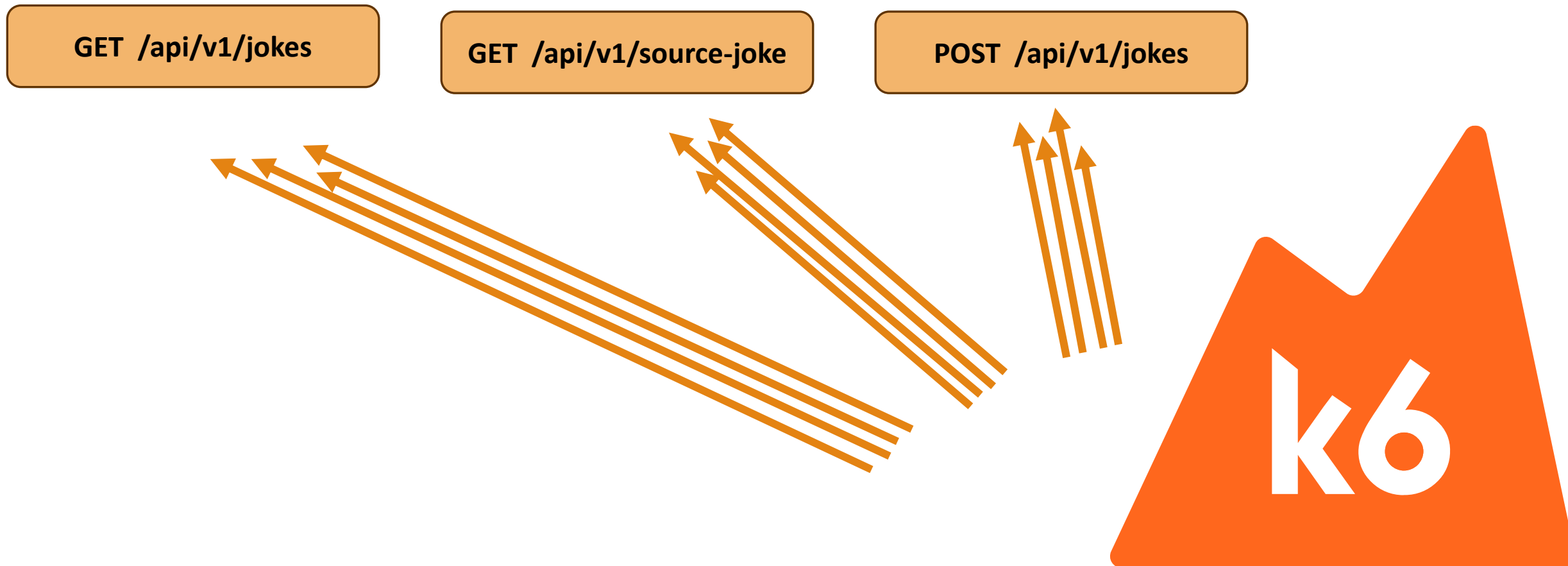
@ArchTest
static final ArchRule servicesShouldBeInServicePackage = classes()
    .that()
    .haveSimpleNameEndingWith("Service")
    .should()
    .resideInAPackage("..service..");

@ArchTest
static final ArchRule servicePackageContainsOnlyServices = classes()
    .that()
    .resideInAPackage("..service..")
    .should()
    .haveSimpleNameEndingWith("Service");
```

Architekturtests - Frontend

- Technologie: fallow-rs, vitest
- Fallow: Statisches code-analyse Tool für JS/TS, mit anpassbaren architektur-guards
- Unit tests erwarten: frontend/src gespiegelt in frontend/test/unit

Load Tests



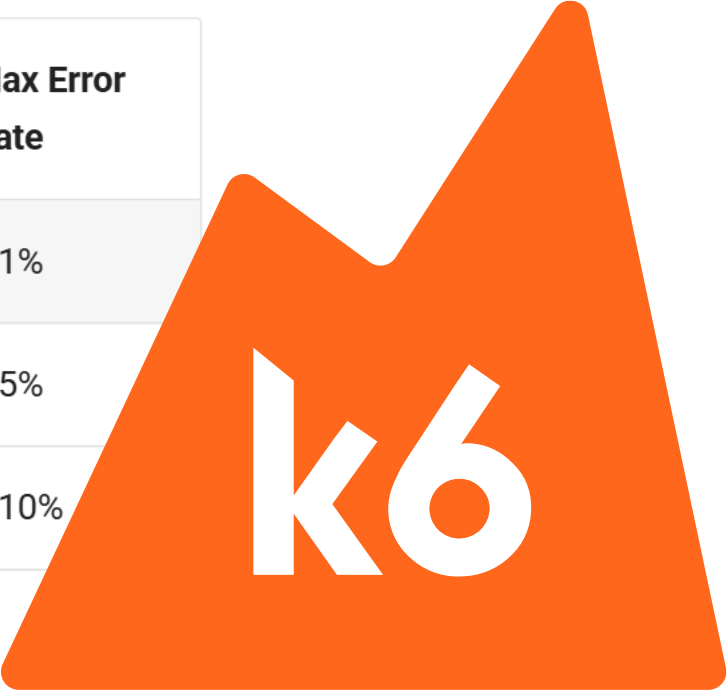
Load Tests

GET /api/v1/jokes

GET /api/v1/source-joke

POST /api/v1/jokes

Scenario	GET VUs	Source VUs	POST VUs	Duration	p95 Latency	Max Error Rate
Baseline	6	1	3	30s	< 200ms	< 1%
Stress	5→20→0	2	2→8→0	2m	< 600ms	< 5%
Spike	0→50→0	1	0→10→0	40s	< 900ms	< 10%

The k6 logo is displayed on a large orange triangle. It consists of the letters 'k' and '6' in a white, bold, sans-serif font.

E2E Tests: Playwright

- Strukturelle Trennung von Locators und Tests

```
test/e2e/pages/login-page.ts

export class LoginPage extends BasePage {
  readonly url = "/login";

  get usernameInput() {
    return this.page.getByTestId("username-input");
  }
  get passwordInput() {
    return this.page.getByTestId("password-input");
  }
  get submitButton() {
    return this.page.getByTestId("submit-login-button");
  }
  //...
  async login(username: string, password: string) {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.submitButton.click();
  }
  // ...
}
```

Page Object Model (POM)

```
test/e2e/tests/login.spec.ts

test.describe('Login page', () => {
  test('displays login form', async ({ page }) => {
    const loginPage = new LoginPage(page);
    await loginPage.navigate();
    await loginPage.verifyTitle();
    await expect(loginPage.usernameInput).toBeVisible();
    await expect(loginPage.passwordInput).toBeVisible();
    await expect(loginPage.submitButton).toBeVisible();
  });

  test('shows validation errors on empty submission', async ({ page }) => {
    const loginPage = new LoginPage(page);
    await loginPage.navigate();
    await loginPage.submitLogin();
    await loginPage.verifyValidationError();
  });
});
```

Playwright test

Security- / Integrations-Tests

- Geschützte Endpunkte testen: *kann ohne Authentication zugegriffen werden?*
- „Angriffsszenarien“:
 - Anfrage ohne Token / Header
 - Anfrage mit ungültigem Token
- Unittests im Backend testen, wie einzelne Bausteine (z. B. Controller) reagieren
- Shell-Skript testet Integration mit HTTP-Anfragen an den laufenden Container

Security-Tests

```
47  @Nested
48  @DisplayName("createJoke")
49  class CreateJoke {
50      @Nested
51      @DisplayName("success scenarios")
52      class Success {
53          @Test
54          @DisplayName("should successfully create joke with valid auth")
55  > void shouldReturn200WhenValidTokenProvided() { ...
73      }
74
75      @Nested
76      @DisplayName("failure scenarios")
77      class Failure {
78          @Test
79          @DisplayName("should fail with 401 Unauthorized when no token provided")
80  > void shouldFailWith401Unauthorized() { ...
96
97          @Test
98          @DisplayName("should fail with 401 when no Authorization header is present (via MVC)")
99  > void shouldFailWith401WhenNoHeaderViaMvc() { ...
114     }
115 }
```

Authentication-
Service gemockt

Integrations-Tests

```
113 # — Stage 3: Healthcheck —————
114 echo ""
115 echo "— Stage 3: Waiting for health endpoint —"
116
117 MAX_RETRIES=30
118 RETRY_INTERVAL=2
119 HEALTH_OK=false
120
121 for i in $(seq 1 $MAX_RETRIES); do
122     HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" "$HEALTH_URL" 2>/dev/null || echo "000")
123
124     if [[ "$HTTP_CODE" == "200" ]]; then
125         HEALTH_OK=true
126         break
127     fi
128
129     echo "  Attempt $i/$MAX_RETRIES - HTTP $HTTP_CODE - retrying in ${RETRY_INTERVAL}s..."
130     sleep "$RETRY_INTERVAL"
131 done
132
133 if [[ "$HEALTH_OK" == "true" ]]; then
134     pass "Health endpoint responded with HTTP 200 after $i attempts"
135 else
136     fail "Health endpoint did not respond with HTTP 200 after $MAX_RETRIES attempts"
137     exit 1
138 fi
```

Startet die
Anwendung
korrekt?

Integrations-Tests

```
140 # — Stage 4: Correct login —————
141 echo ""
142 echo "— Stage 4: Correct login with seed admin credentials —"
143
144 LOGIN_RESPONSE=$(curl -s -w "\n%{http_code}" \
145   -X POST "$LOGIN_URL" \
146   -H "Content-Type: application/json" \
147   -d @- 2>/dev/null <<EOF
148 {"username":"$ADMIN_USERNAME","password":"$ADMIN_PASSWORD"}
149 EOF
150 )
151
152 HTTP_BODY=$(echo "$LOGIN_RESPONSE" | sed '$ d')
153 HTTP_CODE=$(echo "$LOGIN_RESPONSE" | tail -n 1)
154
155 if [[ "$HTTP_CODE" == "200" ]]; then
156   pass "Correct login returned HTTP 200"
157 else
158   fail "Correct login returned HTTP $HTTP_CODE (expected 200)"
159 fi
160
161 # Verify the response body contains a valid token (not null/empty)
162 TOKEN=$(echo "$HTTP_BODY" | sed -n 's/.*"token"[[:space:]]*:[[:space:]]*"([^\"]*)".*/\1/p')
163
164 if [[ -n "$TOKEN" && "$TOKEN" != "null" ]]; then
165   pass "Login token extracted"
166 else
167   fail "Could not extract login token from response body"
168   exit 1
169 fi
```

Kann man sich
echt anmelden?

Security- / Integrations-Tests

```
171 # — Stage 5: Wrong password —————
172 echo ""
173 echo "— Stage 5: Wrong password login attempt —"
174
175 WRONG_RESPONSE=$(curl -s -w "\n%{http_code}" \
176     -X POST "$LOGIN_URL" \
177     -H "Content-Type: application/json" \
178     -d '{"username":"'"$ADMIN_USERNAME"'',"password":"wrong-password-123"}' 2>/dev/null)
179
180 WRONG_CODE=$(echo "$WRONG_RESPONSE" | tail -n 1)
181
182 if [[ "$WRONG_CODE" == "404" ]]; then
183     pass "Wrong password returned HTTP 404 as expected"
184 else
185     fail "Wrong password returned HTTP $WRONG_CODE (expected 404)"
186 fi
```

Falsches
Passwort
erkannt?

Security- / Integrations-Tests

```
203 # — Stage 7: "Penetration" test - protected endpoint without token —
204 echo ""
205 echo "— Stage 7: Penetration test - protected endpoint without token —"
206
207 √ NO_AUTH_RESPONSE=$(curl -s -w "\n%{http_code}" \
208     -X POST "$JOKES_URL" \
209     -H "Content-Type: application/json" \
210     -d '{"content":"Security test joke","externalId":"security-no-auth"}' 2>/dev/null)
211
212 NO_AUTH_CODE=$(echo "$NO_AUTH_RESPONSE" | tail -n 1)
213
214 √ if [[ "$NO_AUTH_CODE" == "401" ]]; then
215     pass "POST /jokes without Authorization header returned HTTP 401"
216 √ else
217     fail "POST /jokes without Authorization header returned HTTP $NO_AUTH_CODE (expected 401)"
218 fi
```

Kein Token!

Fehlerbehandlung im Backend

- Experiment: Either Type
- Bekannt aus Scala
- Fehlerfall im "Left" Case, Success im "Right" Case
- Nahtloses aneinanderschachteln der Eithers

```
backend/src/main/java/com/chucknorris/common/controller/BaseController.java

Either.fromOptional(token, new ErrorResultStatus(401, UUID.randomUUID().toString()))
    .flatMap(authService::checkTokenIsValid)
    .validate(isValid -> isValid, new ErrorResultStatus(401, "Invalid token"))
    .flatMap(r -> action.get());
```

Supply Chain Attacks

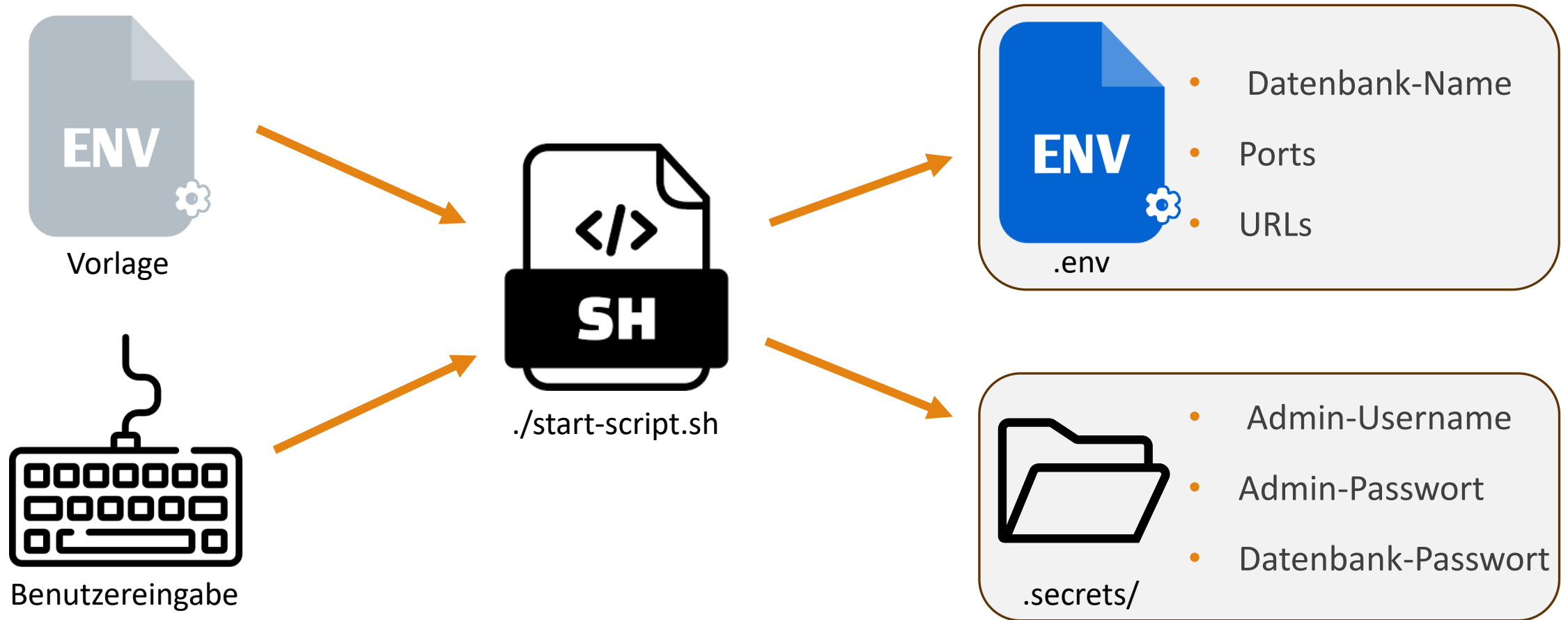
- Verwendung weniger Dependencies mit festen Versionen
- In Frontend, Backend und Github Action Workflows

```
.github/workflows/test-backend.yaml  
  
- name: Set up Java  
  uses: actions/setup-java@fe779bf55ef683c9413269276a7588bb9cedf07a  
  with:  
    java-version: '21'  
    distribution: 'temurin'  
    cache: maven
```

Besonderheiten

- “Either” Type
- npm-Supply-Chain
- Docs:
 - Spell Checking
 - Formatierung von Markdown-Tabellen
 - Automatische Generierung von Grafiken
- Start-/ Endskript
 - Seed-Admin
 - Trennung in .secrets/ und .env
- Statische Code-Analyse inkl. Architekturtests im Frontend

Applikation starten und einrichten



"People say Chuck Norris died in 2026. But that can't be the whole story. Death didn't take Chuck Norris. Chuck Norris just decided it was finally time to train the afterlife."

Bild- und Quellverzeichnis

- <https://www.scala-lang.org/api/3.x/scala/util/Either.html>
- <https://playwright.dev/docs/pom>
- <https://grafana.com/docs/k6/latest/>
- <https://uxwing.com/bash-code-file-black-icon/>
- <https://user-images.githubusercontent.com/45191605/274121951-21239cc4-37b3-4dd2-864f-d5f528da4e22.png>
- https://www.flaticon.com/free-icon/keyboard_689392